Research Notes on Monte Carlo tree search

Monte Carlo tree search

>> Section 1

Principle of operation The focus of MCTS is on the analysis of the most promising moves, expanding the search tree based on random sampling of the search space. The application of Monte Carlo tree search in games is based on many playouts, also called roll-outs. In each playout, the game is played out to the very end by selecting moves at random. The final game result of each playout is then used to weight the nodes in the game tree so that better nodes are more likely to be chosen in future playouts. The most basic way to use playouts is to apply the same number of playouts after each legal move of the current player, then choose the move which led to the most victories. The efficiency of this method—called Pure Monte Carlo Game Search—often increases with time as more playouts are assigned to the moves that have frequently resulted in the current player's victory according to previous playouts. Each round of Monte Carlo tree search consists of four steps:

>> Section 2

Monte Carlo method The Monte Carlo method, which uses random sampling for deterministic problems which are difficult or impossible to solve using other approaches, dates back to the 1940s. In his 1987 PhD thesis, Bruce Abramson combined minimax search with an expected-outcome model based on random game playouts to the end, instead of the usual static evaluation function. Abramson said the expected-outcome model "is shown to be precise, accurate, easily estimable, efficiently calculable, and domain-independent." He experimented in-depth with tic-tac-toe and then with machine-generated evaluation functions for Othello and chess. Such methods were then explored and successfully applied to heuristic search in the field of automated theorem proving by W. Ertel, J. Schumann and C. Suttner in 1989, thus improving the exponential search times of uninformed search algorithms such as e.g. breadth-first search, depth-first search or iterative deepening. In 1992, B. Brügmann employed it for the first time in a Go-playing program. In 2002, Chang et al. proposed the idea of "recursive rolling out and backtracking" with "adaptive" sampling choices in their Adaptive Multi-stage Sampling (AMS) algorithm for the model of Markov decision processes. AMS was the first work to explore the idea of UCB-based exploration and exploitation in constructing sampled/simulated (Monte Carlo) trees and was the main seed for UCT (Upper Confidence Trees).

>> Section 3

Pure Monte Carlo game search This basic procedure can be applied to any game whose positions necessarily have a finite number of moves and finite length. For each position, all feasible moves are determined: k random games are played out to the very end, and the scores are recorded. The move leading to the best score is chosen. Ties are broken by fair coin flips. Pure Monte Carlo Game Search results in strong play in several games with random elements, as in the game EinStein würfelt nicht!. It converges to optimal play (as k tends to infinity) in board filling games with random turn order, for instance in the game Hex with random turn order. DeepMind's AlphaZero replaces the simulation step with an evaluation based on a neural network.

>> Section 4

Improvements Various modifications of the basic Monte Carlo tree search method have been proposed to shorten the search time. Some employ domain-specific expert knowledge, others do not. Monte Carlo tree search can use either light or heavy playouts. Light playouts consist of random moves while heavy playouts apply various heuristics to influence the choice of moves. These heuristics may employ the results of previous playouts (e.g. the Last Good Reply heuristic) or expert knowledge of a given game. For instance, in many Go-playing programs certain stone patterns in a portion of the board influence the probability of moving into that area. Paradoxically, playing suboptimally in simulations sometimes makes a Monte Carlo tree search program play stronger overall.

>> Section 5

wi stands for the number of wins for the node considered after the i-th move ni stands for the number of simulations for the node considered after the i-th move Ni stands for the total number of simulations after the i-th move run by the parent node of the one considered c is the exploration parameter—theoretically equal to 2 {\displaystyle {\sqrt {2}}} ; in practice usually chosen empirically The first component of the formula above corresponds to exploitation; it is high for moves with high average win ratio. The second component corresponds to exploration; it is high for moves with few simulations. Most contemporary implementations of Monte Carlo tree search are based on some variant of UCT that traces its roots back to the AMS simulation optimization algorithm for estimating the value function in finite-horizon Markov Decision Processes (MDPs) introduced by Chang et al. (2005) in Operations Research. (AMS was the first work to explore the idea of UCB-based exploration and exploitation in constructing sampled/simulated (Monte Carlo) trees and was the main seed for UCT.)

>> Section 6

Research Notes on Monte Carlo tree search

Advantages and disadvantages Although it has been proven that the evaluation of moves in Monte Carlo tree search converges to minimax when using UCT, the basic version of Monte Carlo tree search converges only in so called "Monte Carlo Perfect" games. However, Monte Carlo tree search does offer significant advantages over alpha–beta pruning and similar algorithms that minimize the search space. In particular, pure Monte Carlo tree search does not need an explicit evaluation function. Simply implementing the game's mechanics is sufficient to explore the search space (i.e. the generating of allowed moves in a given position and the game-end conditions). As such, Monte Carlo tree search can be employed in games without a developed theory or in general game playing. The game tree in Monte Carlo tree search grows asymmetrically as the method concentrates on the more promising subtrees. Thus, it achieves better results than classical algorithms in games with a high branching factor. A disadvantage is that in certain positions, there may be moves that look superficially strong, but that actually lead to a loss via a subtle line of play. Such "trap states" require thorough analysis to be handled correctly, particularly when playing against an expert player; however, MCTS may not "see" such lines due to its policy of selective node expansion. It is believed that this may have been part of the reason for AlphaGo's loss in its fourth game against Lee Sedol. In essence, the search attempts to prune sequences which are less relevant. In some cases, a play can lead to a very specific line of play which is significant, but which is overlooked when the tree is pruned, and this outcome is therefore "off the search radar".